搜索 2 个关键词，参考 11 篇资料 >

核心一句话：

Runnable 接口里定义的 run () 方法，本身写死了：`public abstract void run();`，它没有 `throws` 任何受检异常！

你重写它的时候，不能私自加多抛出受检异常（`InterruptedException` 就是受检异常），所以不能在 run 上加 `throws InterruptedException`，只能 try-catch 包住 sleep。

大白话

重点一句话：

`RuntimeException` 是运行时异常，不受这个限制！run 本来不能加受检异常，但可以加 RuntimeException。

分开两类异常，很好记：

1. **受检异常（比如 InterruptedException）**
   接口 Runnable 的 run () 没有写 throws 它，重写的时候**不能往外抛**，必须 try-catch。
2. **运行时异常（RuntimeException 和它的子类）**
   它特殊！不管父方法有没有写 throws，你重写的时候都可以声明 throws RuntimeException，也可以不写。编译器不检查它。

运行结果：

```
[小红]制作[卡布奇诺]，吧台当前数量：[1]
[李四]取走[卡布奇诺]，吧台当前数量：[0]
[小明]制作[珍珠奶茶]，吧台当前数量：[1]
[张三]取走[珍珠奶茶]，吧台当前数量：[0]
[小明]制作[珍珠奶茶]，吧台当前数量：[1]
[小红]制作[卡布奇诺]，吧台当前数量：[2]
[李四]取走[珍珠奶茶]，吧台当前数量：[1]
[张三]取走[卡布奇诺]，吧台当前数量：[0]
[小明]制作[珍珠奶茶]，吧台当前数量：[1]
[小红]制作[卡布奇诺]，吧台当前数量：[2]
[小红]制作[卡布奇诺]，吧台当前数量：[3]
[小明]制作[珍珠奶茶]，吧台当前数量：[4]
[张三]取走[珍珠奶茶]，吧台当前数量：[3]
[李四]取走[卡布奇诺]，吧台当前数量：[2]
[小明]制作[珍珠奶茶]，吧台当前数量：[3]
[小红]制作[卡布奇诺]，吧台当前数量：[4]
[张三]取走[卡布奇诺]，吧台当前数量：[3]
[李四]取走[珍珠奶茶]，吧台当前数量：[2]
[小红]制作[卡布奇诺]，吧台当前数量：[3]
[小明]制作[珍珠奶茶]，吧台当前数量：[4]
[小红]制作[卡布奇诺]，吧台当前数量：[5]
[李四]取走[珍珠奶茶]，吧台当前数量：[4]
[张三]取走[卡布奇诺]，吧台当前数量：[3]
[小明]制作[珍珠奶茶]，吧台当前数量：[4]
[小红]制作[卡布奇诺]，吧台当前数量：[5]
[张三]取走[卡布奇诺]，吧台当前数量：[4]
[李四]取走[珍珠奶茶]，吧台当前数量：[3]
[小明]制作[珍珠奶茶]，吧台当前数量：[4]
[小红]制作[卡布奇诺]，吧台当前数量：[5]
[李四]取走[卡布奇诺]，吧台当前数量：[4]
[张三]取走[珍珠奶茶]，吧台当前数量：[3]
[小明]制作[珍珠奶茶]，吧台当前数量：[4]
[小红]制作[卡布奇诺]，吧台当前数量：[5]
```

```
[小红]下班了
[李四]取走[卡布奇诺]，吧台当前数量：[4]
[张三]取走[珍珠奶茶]，吧台当前数量：[3]
[小明]制作[珍珠奶茶]，吧台当前数量：[4]
[小明]下班了
[李四]取走[卡布奇诺]，吧台当前数量：[3]
[张三]取走[珍珠奶茶]，吧台当前数量：[2]
[张三]取走[卡布奇诺]，吧台当前数量：[1]
[李四]取走[珍珠奶茶]，吧台当前数量：[0]
[李四]走了
[张三]走了
```

代码：

```
package Bao8;

public class Main {
    public static void main(String[] args) throws RuntimeException {
        //创建员工和顾客
        Staff staff1 = new Staff("珍珠奶茶","小明");
        Staff staff2 = new Staff("卡布奇诺","小红");
        Customer customer1 = new Customer("张三");
        Customer customer2 = new Customer("李四");
        //创建线程
        new Thread(staff1).start();
        new Thread(customer1).start();
        new Thread(customer2).start();
        new Thread(staff2).start();
    }
}
```

```
package Bao8;

import java.util.ArrayList;
import java.util.List;

public class Bar {
    //吧台容器
    private  volatile List<Staff> milkTeas =new ArrayList<>();
    //吧台实例，用静态避免多个线程创建的不一样
    private static Bar bar = new Bar();
    //创建方法加入奶茶
    public synchronized void  Put(Staff staff) {
        //吧台最多5杯奶茶
        while(milkTeas.size()>=5){
            try {
                //等待
                wait();
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
```

```
        milkTeas.add(staff);
        System.out.println("["+staff.getName()+"]制作["+staff.getMilkTea()+"]，吧台当前数量:
["+milkTeas.size()+"]");
        notifyAll();
    }

    //创建方法取出奶茶
    public synchronized void take(Customer customer)  {
        //吧台有奶茶时，顾客才能取
        while(milkTeas.size()<=0) {
            try {
                //等待
                wait();
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
            //顾客取走奶茶
           Staff staff = milkTeas.remove(0);
            System.out.println("[" + customer.getName() + "]取走[" + staff.getMilkTea() +
"]，吧台当前数量:[" + milkTeas.size() + "]");
        notifyAll();
    }

//获得对象
    public static Bar getInstance(){
        return bar;
    }
}
```

```
package Bao8;

import java.util.ArrayList;
public class Staff implements Runnable {
    private String name;
    private String milkTea;
    //构造方法
    public Staff(String milkTea, String name) {
        this.milkTea = milkTea;
        this.name = name;
    }

    @Override
    public void run() throws RuntimeException  {
        for(int i=1;i<=10;i++){
            //员工制作奶茶
            Bar.getInstance().Put(this);
            try {
                //制作时间
                Thread.sleep(999);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
```

```
            }
            //员工下班
            if(i==10){
                System.out.println("["+name+"]下班了");
            }
        }
    }

    public String getMilkTea() {
        return milkTea;
    }

    public void setMilkTea(String milkTea) {
        this.milkTea = milkTea;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```
package Bao8;

public class Customer implements Runnable {
    private String name;

    public Customer(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
    //顾客取奶茶
    @Override
    public void run() throws RuntimeException {
        for (int i = 1; i <= 10; i++) {
            Bar.getInstance().take(this);
            try {
                //顾客取奶茶时间
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
```

```
            if(i==10){
                System.out.println("["+name+"]走了");
            }
        }
    }
}
```